

AUTOGEST

Guia de ambiente de desenvolvimento e registro de alterações

Instalação no Windows, execução no Android, integração Flutter + C por FFI, testes e pendências conhecidas.

Repositório	github.com/upereira-jp/autogest
Branch validada	teste-pra-rodar
Commit-base	d6fa046 - C totalmente integrado ao flutter via FFI
Plataforma do MVP	Android - arm64-v8a e x86_64
Data deste guia	22 de junho de 2026

Resultado atual

O aplicativo foi compilado e executado no Android; os dados permaneceram no SQLite; os cálculos e alertas foram produzidos pelo núcleo C; os testes Flutter e C passaram; e o analisador Dart terminou sem problemas.

PARTE I - PREPARAR E EXECUTAR

1. Visão rápida para um novo integrante

Esta página é o caminho curto. As seções seguintes explicam cada item com mais detalhes e ajudam a resolver erros.

O que é obrigatório para apenas rodar o app

Item	Por que o projeto precisa
Git for Windows	Clonar e atualizar o repositório.
Flutter SDK - canal stable	Precisa fornecer Dart $\geq 3.12.2$ e $< 4.0.0$.
Android Studio	Fornece Android SDK, emulador, JDK e gerenciamento das ferramentas nativas.
Android SDK tools	Platform-Tools, Command-line Tools, Build-Tools e ao menos uma plataforma Android recente.
NDK e CMake	O Gradle usa CMake para compilar native/ como libautogest.so.
Dispositivo Android	Emulador x86_64 ou celular arm64-v8a com depuração USB.

Não é obrigatório para rodar

LLVM/Clang é necessário somente para regenerar autogest_bindings.dart. GCC e um CMake no PATH são necessários somente para testar o C isoladamente no Windows. O binding gerado já está versionado no Git.

Comandos do primeiro uso

```
git clone https://github.com/upereira-jp/autogest.git
cd autogest
git fetch origin
git switch --track origin/teste-pra-rodar
flutter pub get
flutter doctor -v
flutter devices
flutter run -d <ID_DO_ANDROID>
```

Escolha Android

Não execute esta branch em Windows ou Chrome. FfiService abre libautogest.so e, por enquanto, está implementado somente para Android.

PARTE I - PREPARAR E EXECUTAR

2. Instalação detalhada no Windows

2.1 Git

- Baixe o Git for Windows em git-scm.com/download/win.
- Aceite as opções padrão do instalador; mantenha o Git disponível no PATH.
- Abra um novo PowerShell e valide com `git --version`.
- Configure nome e e-mail se ainda não tiver feito isso.

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

2.2 Flutter SDK

- Siga o instalador oficial para Windows em docs.flutter.dev/install/manual.
- Use o canal stable. O `pubspec.yaml` exige Dart 3.12.2 ou superior, mas menor que 4.0.0.
- Extraia o SDK em uma pasta simples, por exemplo `C:\src\flutter`. Evite Program Files por causa de permissões.
- Adicione `C:\src\flutter\bin` ao PATH do usuário e abra um novo terminal.
- Valide a instalação.

```
flutter --version
dart --version
flutter doctor -v
```

Pasta do projeto

Prefira clonar em um caminho curto, sem acentos e fora de pastas sincronizadas, por exemplo `C:\dev\autogest`. OneDrive pode funcionar, mas aumenta a chance de bloqueios de arquivos e caminhos longos.

2.3 Android Studio e componentes Android

- Instale o Android Studio pela página oficial: developer.android.com/studio/install.
- No assistente inicial, mantenha Android SDK, Android SDK Platform e Android Virtual Device.
- Abra Settings > Languages & Frameworks > Android SDK.
- Na aba SDK Platforms, instale uma plataforma Android recente compatível com a versão indicada pelo Flutter.
- Na aba SDK Tools, marque os itens da tabela abaixo e aplique as alterações.

SDK Tool	Uso
Android SDK Build-Tools	Compila e empacota o APK.
Android SDK Command-line Tools (latest)	Permite que o Flutter gerencie SDK e licenças.
Android SDK Platform-Tools	Inclui adb para detectar celular e emulador.
Android Emulator	Necessário se não for usado um celular físico.
NDK (Side by side)	Compila os arquivos C para Android.
CMake	Processa <code>native/CMakeLists.txt</code> durante o build.

O projeto define `ndkVersion = flutter.ndkVersion`. Se o Gradle informar uma versão de NDK ausente, volte ao SDK Manager, ative Show Package Details e instale exatamente a versão solicitada.

```
flutter doctor --android-licenses
flutter doctor -v
```

PARTE I - PREPARAR E EXECUTAR

3. Preparar um dispositivo Android

Opção A - Emulador

- Ative virtualização na BIOS/UEFI se o Android Studio solicitar.
- Abra Tools > Device Manager > Create Virtual Device.
- Escolha um aparelho e uma imagem Android x86_64. Essa arquitetura está incluída no APK do projeto.
- Inicie o emulador e aguarde a tela inicial do Android.

Opção B - Celular físico

- No celular, habilite Opções do desenvolvedor e Depuração USB.
- Conecte o cabo USB e aceite a autorização RSA exibida no aparelho.
- A maioria dos aparelhos atuais usa arm64-v8a, arquitetura incluída no projeto.

Confirmar detecção

```
adb devices  
flutter devices
```

Se nenhum Android aparecer

Reinicie o emulador; troque o cabo ou a porta USB; aceite a autorização no celular; confira o driver USB do fabricante; e execute `adb kill-server` seguido de `adb start-server`.

IDE para editar o projeto

- VS Code: instale as extensões oficiais Flutter e Dart.
- Android Studio: instale o plugin Flutter; o plugin Dart será incluído como dependência.
- Use apenas uma pasta-raiz: abra a pasta que contém `pubspec.yaml`.
- Se o Windows pedir suporte a links simbólicos, ative Settings > System > For developers > Developer Mode.

Arquivos locais que não devem ir para o Git

- `android/local.properties`: guarda caminhos locais do Android SDK e Flutter.
- Pastas `build/` e `.dart_tool/`: são recriadas automaticamente.
- Configurações pessoais da IDE e credenciais não devem ser compartilhadas.

PARTE I - PREPARAR E EXECUTAR

4. Clonar, executar e validar

4.1 Clonar a branch validada

```
cd C:\dev
git clone https://github.com/upereira-jp/autogest.git
cd autogest
git fetch origin
git switch --track origin/teste-pra-rodar
git status
```

Em uma clonagem nova, o comando com `--track` cria a branch local ligada a branch remota. Se ela já existir localmente, use apenas `git switch teste-pra-rodar`.

4.2 Baixar dependências Dart

```
flutter clean
flutter pub get
```

O `flutter clean` é útil quando uma máquina recebeu artefatos antigos. Ele não precisa ser executado em todo build.

4.3 Verificacoes antes de abrir o app

```
flutter analyze
flutter test
flutter devices
```

- Esperado em `flutter analyze`: No issues found!
- Esperado em `flutter test`: All tests passed!
- `flutter devices` deve listar pelo menos um Android.

4.4 Executar e gerar APK

```
flutter run -d <ID_DO_ANDROID>

# APK de desenvolvimento
flutter build apk --debug
```

O APK de debug será gerado normalmente em `build/app/outputs/flutter-apk/app-debug.apk`. O Gradle chama o CMake automaticamente e inclui `libautogest.so`.

4.5 Teste manual de ponta a ponta

Passo	Resultado esperado
1	Cadastre abastecimento: 40 L, R\$ 200, km 1000.
2	Cadastre abastecimento: 40 L, R\$ 240, km 1400.
3	Confirme consumo do segundo trecho: 10,0 km/L.
4	Cadastre uma manutenção com intervalo em dias.
5	Confirme o alerta na aba Manutencoes e na tela inicial.
6	Feche com q no terminal, execute novamente e confirme a persistencia.

PARTE I - PREPARAR E EXECUTAR

5. Ferramentas opcionais: bindings e testes C

5.1 Regenerar bindings com LLVM/Clang

Somente quando autogest.h mudar

Não execute ffigen por rotina. lib/src/ffi/autogest_bindings.dart já está no repositório e deve ser regenerado apenas quando o contrato C for alterado.

- Instale LLVM/Clang para Windows em releases.llvm.org/download.html.
- A configuração atual espera LLVM em C:\Program Files\LLVM.
- O PATH do sistema não é necessário para ffigen porque llvm-path aponta diretamente para a instalação.
- O include atual usa o caminho curto C:\PROGRA~1\LLVM\lib\clang\22\include para evitar que o espaço de Program Files seja dividido.
- Se a versão instalada não for 22, ajuste esse trecho em ffigen.yaml para a pasta real.

```
dart run ffigen --config ffigen.yaml
Test-Path lib/src/ffi/autogest_bindings.dart
```

O resultado esperado do segundo comando é True. Avisos sobre constantes privadas iniciadas por sublinhado e tamanho de enum não impediram a geração.

5.2 Testar o C isoladamente no Windows

- Instale um compilador C. O fluxo validado usou GCC/MinGW; MSYS2 é uma opção prática: msys2.org.
- Instale CMake e deixe gcc e cmake disponíveis no PATH.
- Use uma pasta de build separada da usada pelo Flutter.

```
gcc --version
cmake --version

cmake -S native -B build/native-host -G "MinGW Makefiles"
cmake --build build/native-host
ctest --test-dir build/native-host --output-on-failure
```

Checagem rápida de sintaxe C

```
gcc -std=c11 -Wall -Wextra -Wpedantic -I native -fsyntax-only `
  native/src/inputs.c `
  native/src/validacao.c `
  native/src/calculos.c `
  native/src/alertas.c
```

PARTE II - O QUE FOI ALTERADO

6. Arquitetura depois da integração

O projeto agora separa persistência e regras de calculo. O SQLite permanece no Dart; o C recebe copias temporárias dos registros para calcular consumo, gastos e alertas.

Interface Flutter	
Telas, formularios e estado	
Providers	
Persistência	Cálculos
DAOs -> sqflite -> SQLite	AutogestService -> FfiService
Dados salvos no aparelho	Bindings -> libautogest.so -> C

Fluxo de memória no FFI

- FfiService converte DateTime para segundos Unix e copia campos Dart para structs C alocadas com calloc.
- As funções C processam arrays e retornam valores, structs ou uma lista dinâmica de alertas.
- Arrays criados no Dart são liberados com calloc.free em blocos finally.
- A lista de alertas criada com malloc no C é liberada pela função liberar_lista_alertas.
- Strings de combustível e descrição ficam zeradas no envio atual porque os cálculos usados não as consultam.

Decisões de contrato

Decisão	Implementação
Datas públicas	int64_t em segundos Unix. time_t fica apenas local para localtime().
Banco de dados	Somente Dart/sqflite. O SQLite duplicado no C foi removido.
Biblioteca Android	libautogest.so, gerada pelo CMake via Gradle.
Categorias mensais	Coluna 0 = combustível; colunas seguintes = tipos de manutencao.
Plataformas	arm64-v8a para celulares e x86_64 para emuladores.

PARTE II - O QUE FOI ALTERADO

7. Registro completo dos arquivos alterados

A tabela corresponde ao commit d6fa046: 17 arquivos alterados, 159 inserções e 142 remoções segundo o Git.

Arquivo	Situação	Alteração
analysis_options.yaml	Modificado	Exclui o binding gerado da análise para evitar avisos de constantes internas do ffigen.
android/app/build.gradle.kts	Modificado	Ativa NDK para arm64-v8a/x86_64 e aponta externalNativeBuild para native/CMakeLists.txt.
ffigen.yaml	Modificado	Define o LLVM em C:\Program Files\LLVM e o include do Clang 22 pelo caminho curto.
lib/main.dart	Modificado	Remove MockService do provider e cria FfiService como implementação real do AutogestService.
lib/models/resumo_gastos.dart	Modificado	Adiciona gastoPorMesCategoria e a matriz 12 x categorias no resumo zerado.
lib/services/ffi_service.dart	Modificado	Implementa carregamento da .so, conversão de structs, todas as chamadas e liberacao de memoria.
lib/src/ffi/autogest_bindings.dart	Adicionado	Binding Dart gerado automaticamente a partir de native/autogest.h. Não editar manualmente.
native/CMakeLists.txt	Modificado	Cria projeto C11, biblioteca compartilhada, exports Windows e testes CTest fora do Android.
native/Test/test_autogest.c	Adicionado	Testa validação, datas, consumo, listas, alertas, resumo e matriz mensal/categoria.
native/autogest.h	Modificado	Padroniza datas em int64_t, define categorias, adiciona matriz e ajusta assinaturas FFI.
native/src/alertas.c	Modificado	Troca cálculos de data para int64_t e mantém alocação/liberacao da lista de alertas.
native/src/banco.c	Removido	Elimina a tentativa de persistência SQLite duplicada no núcleo C.
native/src/calculos.c	Modificado	Remove dependencia SQLite, adapta localtime e preenche gastos por mês e categoria.
native/src/inputs.c	Modificado	Alinha construtores de Abastecimento e Manutenção ao contrato int64_t.
native/src/validacao.c	Modificado	Corrige KM, datas, valores e string nula; remove dependencia SQLite.
native/vedor/sqlite3.c	Removido	Remove arquivo vendorizado vazio/dispensavel.
native/vedor/sqlite3.h	Removido	Remove header vendorizado vazio/dispensavel.

PARTE II - O QUE FOI ALTERADO

8. Alterações técnicas em detalhe

8.1 Uma única persistência

- native/src/banco.c e os arquivos SQLite em native/vedor foram removidos.
- Os DAOs Dart continuam responsáveis por INSERT e SELECT usando sqflite.
- O C ficou focado em regras de cálculo, validação e alertas.
- Isso evita dois bancos, dois esquemas e sincronização duplicada entre Dart e C.

8.2 ABI de datas previsível

- Campos e parâmetros de data públicos passaram de time_t para int64_t.
- O Dart envia millisecondsSinceEpoch dividido por 1000.
- O C usa time_t somente em variáveis locais ao chamar localtime().
- O retorno de alerta é convertido de segundos para milissegundos no Dart.

8.3 Biblioteca compartilhada no Android

- CMake 3.18 ou superior e C11 foram declarados explicitamente.
- A biblioteca autogest é SHARED; no Android vira libautogest.so.
- O Gradle compila a biblioteca junto com o APK por externalNativeBuild.
- Os fontes incluídos são inputs.c, validacao.c, calculos.c e alertas.c.

8.4 FfiService implementado

Método Dart	Ligação com o C
resumoGastos	Totais, 12 meses, categorias e matriz mês/categoria.
consumoMedioGeral	Variação total de km dividida pelos litros consumidos.
kmPorLitroTrecho	Diferença de hodômetro dividida pelos litros do registro atual.
obterKmAtual	Maior hodômetro da lista.
custoPorKm	Soma de gastos no intervalo dividida pela distancia.
gastoPeriodo	Soma abastecimentos dentro do intervalo de datas.
gerarAlertas	Converte lista, chama C, copia alertas e libera a memória nativa.

8.5 Matriz de gastos

ResumoGastos ganhou gasto_por_mes_categoria[12][TOTAL_CATEGORIAS_GASTO]. A coluna 0 representa combustível; cada tipo de manutenção usa índice do enum + 1. O mesmo formato foi reproduzido no modelo Dart.

PARTE II - O QUE FOI ALTERADO

9. Testes e evidências de funcionamento

Camada	Verificacao	Resultado
Compilação sintática C	gcc -std=c11 -Wall -Wextra -Wpedantic -fsyntax-only	Aprovada
Build C/CMake	cmake --build	Aprovado
Testes C	ctest --output-on-failure	Todos passaram
Build Android	flutter build apk --debug	APK gerado
Análise Dart	flutter analyze	No issues found
Testes Flutter	flutter test	All tests passed
FFI em execução	flutter run em Android	libautogest.so carregada
Teste de consumo	km 1000 -> 1400; 40 L	10,0 km/L
Alertas	Manutenção vencida/proxima/em dia	Classificação exibida
Persistência	Fechar e abrir o app	Dados preservados

Cobertura do teste nativo

- KM crescente, decrescente e negativo.
- Valor zero permitido e valor negativo recusado.
- Litros e intervalo de dias invalidos.
- String nula, vazia e preenchida.
- Data atual, futura e timestamp zero.
- Consumo por trecho, consumo geral, KM atual, gasto por período e custo por KM.
- Alertas vencido, próximo e em dia, incluindo memória da lista.
- Totais e distribuição mensal/categoria no ResumoGastos.

Limite dos testes atuais

Os testes Flutter existentes usam o serviço mock em isolamento. A integração FFI completa foi validada manualmente no Android. Um futuro teste de integração automatizado em emulador aumentaria a segurança.

PARTE III - MANUTENÇÃO DO AMBIENTE

10. Erros comuns e como resolver

Mensagem	Causa provável	Ação
rg não é reconhecido	ripgrep não está instalado.	Use Select-String no PowerShell ou instale ripgrep; ele não é exigência do projeto.
Terminal mostra >>	O PowerShell aguarda o restante de um comando multilinha.	Complete as linhas com crase ou pressione Ctrl+C e digite novamente.
Couldn't find dynamic library	ffigen não encontrou LLVM.	Confirme llvm-path em ffigen.yaml e a pasta C:\Program Files\LLVM.
stdint.h file not found	O caminho do include foi dividido no espaço de Program Files.	Use C:\PROGRA~1\LLVM\lib\clang\22\include ou ajuste para sua versão.
Avisos __INT_LEAST...	Constantes internas geradas pelo Clang não são usadas.	Não edite o binding. O arquivo está excluído do flutter analyze.
FfiService apenas no Android	Windows ou Chrome foi selecionado.	Execute flutter devices e escolha um ID Android.
libautogest.so não abre	Build nativo ausente ou ABI não suportada.	Use emulador x86_64/celular arm64, flutter clean, flutter pub get e novo build.
NDK/CMake ausente	Componentes não instalados no Android SDK.	Instale NDK Side by side e CMake pelo SDK Manager; aceite licenças.
No devices found	Emulador desligado ou USB não autorizado.	Ligue o AVD ou aceite a chave RSA no celular; verifique adb devices.
LF will be replaced by CRLF	Normalização de quebra de linha no Windows.	É um aviso do Git, não uma falha de compilacao.
Dart SDK incompatível	Flutter antigo para o pubspec.	Atualize o Flutter stable e confirme Dart >= 3.12.2 e < 4.0.0.

Regra de ouro

Copie o erro completo desde a primeira linha. Informe o comando executado, a plataforma selecionada e a saída de flutter doctor -v. Isso evita diagnosticos por tentativa e erro.

PARTE III - MANUTENÇÃO DO AMBIENTE

11. Pendências e incoerências conhecidas

O MVP roda, mas estes pontos não devem ser tratados como decisões definitivas. Estão registrados para o grupo discutir antes da entrega final.

Prioridade	Tema	Recomendação
Alta	Custo por KM	DashboardProvider chama custoPorKm com kmInicio = 0. Isso pode diluir o valor pela quilometragem total do hodometro. Definir a regra: usar menor KM registrado, distância entre abastecimentos ou outro intervalo.
Alta	Validação duplicada	Formularios validam no Dart, mas as funções validar_* do C ainda não são chamadas pelo FFI. Se a disciplina exige regras no C, expor essas validacoes no servico.
Média	Somente Android	FfiService lança UnsupportedError em Windows, web e outras plataformas. Manter o escopo Android documentado ou implementar bibliotecas por plataforma.
Média	Gráficos	A aba ainda mostra Histórico de consumo - em breve. A matriz de gastos já existe, mas a interface não a utiliza.
Média	Edição e exclusão	A persistência foi desenhada como imutavel: a UI faz INSERT e SELECT. Não existe fluxo geral de UPDATE/DELETE.
Média	Release Android	O build release usa chave de debug. Antes de distribuir fora da turma, configurar applicationId próprio e assinatura de release.
Baixa	Comentarios antigos	ffi_service.dart e main.dart ainda possuem comentarios dizendo que FFI é futura ou que o mock será trocado. Atualizar para refletir o estado real.
Baixa	Metadados do app	pubspec.yaml e applicationId ainda usam nomes padrão de projeto Flutter.
Baixa	ffigen local	O caminho do Clang 22 é específico do Windows. Documentar/parametrizar se o grupo usar outra versão ou sistema.

Ordem recomendada para a próxima iteração

- 1. Definir e corrigir a regra de custo por KM; adicionar teste com exemplo aceito pelo grupo.
- 2. Decidir se as validacoes dos formularios precisam obrigatoriamente passar pelo C.
- 3. Limpar comentarios e nomes padrão; atualizar o documento técnico entregue ao professor.
- 4. Implementar gráficos e, se fizer parte do escopo, edição/exclusao.
- 5. Preparar identificador e assinatura somente quando houver necessidade de APK de release.

PARTE III - MANUTENÇÃO DO AMBIENTE

12. Checklists para o grupo

Primeira configuração de uma máquina

- [] Git instalado e configurado
- [] Flutter stable no PATH e Dart compatível
- [] Android Studio instalado
- [] SDK Platform, Build-Tools, Platform-Tools e Command-line Tools instalados
- [] NDK Side by side e CMake instalados
- [] Licenças aceitas com flutter doctor --android-licenses
- [] flutter doctor -v sem bloqueios para Android
- [] Emulador x86_64 ou celular arm64-v8a detectado
- [] Branch teste-pra-rodar selecionada
- [] flutter pub get concluído
- [] flutter analyze e flutter test aprovados
- [] flutter run abriu o app no Android

Antes de enviar uma alteração

- [] Confirmar a branch atual com git branch --show-current
- [] Rodar dart format nos arquivos Dart alterados
- [] Rodar flutter analyze
- [] Rodar flutter test
- [] Se o C mudou, rodar os testes nativos
- [] Se autogest.h mudou, regenerar bindings e revisar o diff
- [] Rodar no Android e fazer um teste manual curto
- [] Conferir git diff --check e git status --short
- [] Fazer commit com mensagem objetiva e push da branch

Comandos de referencia

```
git branch --show-current
git status --short
dart format lib test
flutter analyze
flutter test
flutter build apk --debug
git diff --check
```

PARTE III - MANUTENÇÃO DO AMBIENTE

13. Referências oficiais

Fonte	Endereço
Flutter - instalação manual	https://docs.flutter.dev/install/manual
Flutter - configuração Android	https://docs.flutter.dev/platform-integration/android/setup
Android Studio - instalação	https://developer.android.com/studio/install
Android - instalar NDK e CMake	https://developer.android.com/studio/projects/install-ndk
Git for Windows	https://git-scm.com/download/win
ffigen no pub.dev	https://pub.dev/packages/ffigen
LLVM releases	https://releases.llvm.org/download.html
CMake downloads	https://cmake.org/download/
MSYS2	https://www.msys2.org/
Repositório AutoGest	https://github.com/upereira-jp/autogest

Sobre versões

Ferramentas Android e Flutter evoluem. Este guia evita fixar uma versão de Flutter e orienta usar o canal stable que cumpra o pubspec. As versões de Gradle, Android Gradle Plugin e Kotlin já estão fixadas nos arquivos do repositório; não as atualize apenas porque uma IDE sugeriu, sem testar o projeto inteiro.

Marco documentado

Ambiente reproduzível e núcleo C integrado

Este documento descreve o estado validado da branch teste-pra-rodar no commit d6fa046. Se o grupo alterar o contrato C, as dependências Android ou o fluxo de dados, atualize também este guia.